


Designed by Remus Teodorescu 2026

LECTURE 11.1

# Vision-Based Navigation

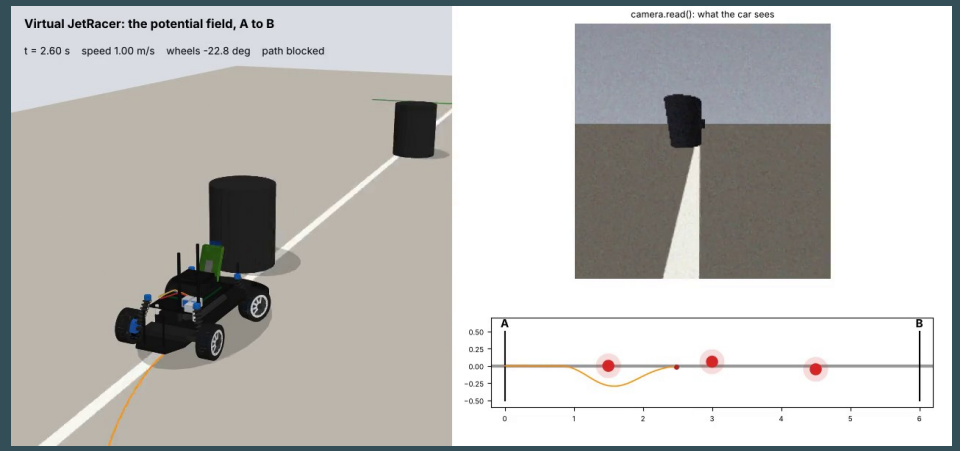

**Remus Teodorescu**

*With support from Research Assistant Noman Khan - nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk


Deep Learning for Engineering · 2026

NEED · RULES · NETWORKS




*Cars, warehouse robots, underwater vehicles and humanoid robots steer by what a camera sees, answering two questions: where is the path, and how do I steer onto it? The lecture answers both with fixed rules, then with a convolutional network and the GPU that makes it fast enough.*

# Learning Objectives

## SECTIONS, IN ORDER

1. **The need** — vehicles and robots that steer by camera
2. **Conventional vision** — fixed rules, and where they fail
3. **Deep learning** — a network that steers, its data, its GPU

*Two questions run through the lecture: where is the path, and how do I steer onto it? Each is answered first in words, then in mathematics - by fixed rules, then by a network - on Ex_11's virtual JetRacer, where the true position of the line is known.*

## READ, AND WHAT EACH ANSWERS

| | |
|---|---|
| **UDL ch. 1** | where is deep learning applied? |
| **UDL ch. 10** | why do images need convolutions? |
| **UDL ch. 8** | what must the training data show? |

# Recap of L10.2 Electrolysers

## 1 – 2 · A NETWORK FOR THE FUEL CHANNEL

**1 How can one network give the steam, the temperature and the current along the channel at every operating point?**

It takes the position, the cell voltage and the inlet temperature and returns the three fields, trained over the whole operating range until the three laws hold. The inlet values are built in. Any operating point is then read off, not solved again. *Go deeper: the network and its inputs (Ex_10.2 section 5); Liu ch. 2.2.*

**2 Why does the local current make the channel harder to solve, and how is the network checked?**

The current has no equation of its own: at every point it is the root of the charge transfer at the local steam fraction and temperature. The network takes it as a third residual, in units of 50 mV. Against a reference solver it was within 0.0022 and 0.19 K. *Go deeper: the coupled laws (Ex_10.2 sections 3 – 5); the comparison with finite volumes.*

## 3 – 4 · THE OPTIMAL VOLTAGE

**3 How is the optimal cell voltage found with the trained network?**

The cost of an hour, the electricity less the hydrogen plus the ageing, is read from the fields, with the 10 K limit as a penalty. Autograd gives its derivative in the voltage, and gradient descent finds 1.250 V in seconds; a sweep of 81 solver runs agrees. *Go deeper: with and without the limit (Ex_10.2 section 6); Liu ch. 5.*

**4 Why does a network help to plan the cell's operation over its lifetime, and what should not be claimed?**

Over a lifetime the decision is a path of voltage and temperature, and a solver's gradient costs one solve per variable. A network gives every gradient by autograd. The ageing constants are assumed, so a predicted lifetime ranks schedules; it is not a promise. *Go deeper: Liu ch. 5; the spread of published ageing rates.*

# Autonomous Vehicles and Robots

**Vision-based navigation** — steering a vehicle or a robot from what its camera sees: where the free path is, where it leads, and what is in the way. The camera is cheap and carries the most information of any sensor, but turning its pixels into a decision is the hard part.

## FOUR OF THEM, AND THE LOOP THEY SHARE



*robotaxi*




*warehouse AGV*



*humanoid*



*underwater vehicle*

**see** one frame → **decide** path, obstacles → **act** steer, speed → **in time** fast enough

*Photographs: Wikimedia Commons. This lecture is the path; L11.2 is what blocks it.*

## WHAT MAKES THE CAMERA'S JOB HARD

| | specific challenges | beside the camera |
|---|---|---|
| road car | night, rain, low sun; 3.6 m blind per 0.1 s | radar, LiDAR, GNSS |
| robotaxi (Cybercab) | no wheel or pedals: no driver to take over | none: cameras only |
| warehouse AGV | shiny floors, narrow aisles, people and forklifts | safety LiDAR, floor markers |
| underwater vehicle | light gone within metres; murky water | sonar, Doppler log, inertial |
| humanoid robot | camera sways; stairs need depth | depth camera, joint angles |

# The JetRacer Pro

**JetRacer Pro** — a small car that computes on board: a Jetson Nano runs the network on its GPU, a wide camera is its only sensor, a servo steers and a brushed motor drives. It is the scaled-down case of the machines on the previous slide, with the same loop: see, decide, act.

## WHERE A ROW OF PIXELS LANDS ON THE FLOOR

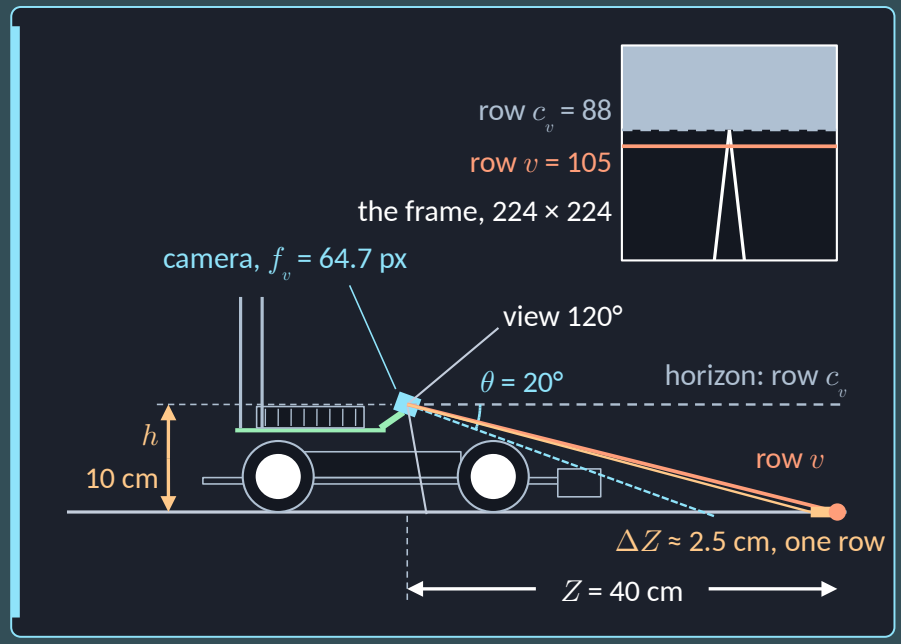
$$v - c_v = \frac{f_v\, h}{Z}, \qquad \Delta Z \approx \frac{Z^2}{f_v\, h}$$

$v$ the image row, $c_v$ the horizon's row, $f_v$ the focal length in pixels
$h$ the camera's height, $Z$ the distance ahead, $\Delta Z$ the floor one row covers

## THE VERIFIED SPECIFICATION

| compute | Jetson Nano 4 GB, 128-core Maxwell GPU |
|---|---|
| camera | IMX219-160: 8 MP, 160° lens; the network reads 224 × 224 |
| steering | Ackermann, 6 kg·cm servo; 4WD, RC380 brushed motor |
| pack | 8.4 V, four 18650 cells, 2S2P; no encoder |

## THE CAMERA ON THE CAR, SIDE VIEW




*To scale. The camera's height, tilt and field of view are the virtual JetRacer's, assumed; they are measured on the car.*

# Vision by Hand-Written Rules

**Conventional vision** — fixed steps chosen by an engineer. The white tape reflects more light than the dark floor, so pixels brighter than the floor mark the line; the centre of the bright run is where the tape crosses the row - the point to steer toward. Nothing is learned from data.

## THE DETECTOR, AS EX_11 WRITES IT

$$w_j = \max\left(I_j - \bar{I} - 0.15,\ 0\right), \qquad \hat{x} = \frac{\sum_j w_j x_j}{\sum_j w_j}$$

$I_j$ the brightness of pixel $j$, $\bar{I}$ the floor's median, $x_j$ its column
$w_j$ its weight, $\hat{x}$ the estimated column of the line

### FROM A FRAME TO A COMMAND

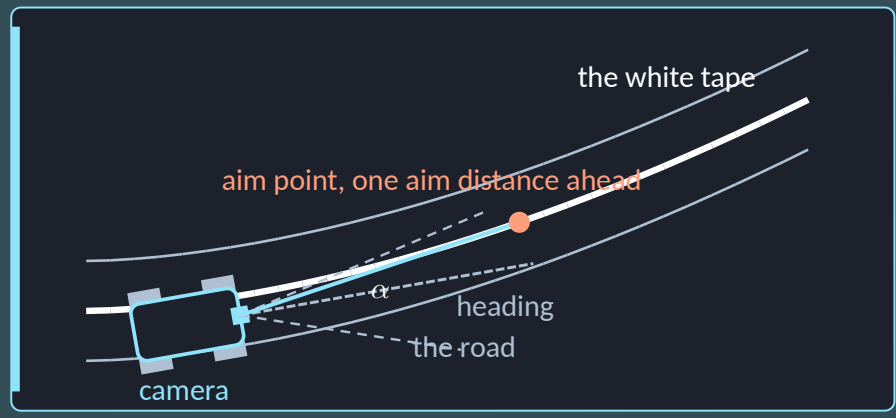
**frame** grey pixels → **threshold** set by hand → **line** bright centre → **rule** steer to it

every step has a number chosen by an engineer, for one floor and one light

## THE STEERING RULE

$$\delta = \arctan\frac{2L\sin\alpha}{\ell_d}$$

$\delta$ the steering angle, $\alpha$ the angle to the aim point
$L$ the wheelbase, 0.20 m, $\ell_d$ the aim distance, 0.40 m




*The rule is pure pursuit: steer onto the circle through the aim point; $L$ and $\ell_d$ assumed until measured.*

# Where Conventional Vision Fails

**Robustness** — keeping the error within what was agreed when the scene changes: the light, a reflection, a shadow, a new floor, an unseen object. A hand-tuned rule is robust only to the changes its designer thought of.

## WHY THE ESTIMATED AIM MOVES TOWARD THE GLARE

$$\hat{x} = \frac{W_\ell x_\ell + W_g x_g}{W_\ell + W_g}$$

$W_\ell, x_\ell$, $W_g, x_g$ the summed weight and mean column of the bright pixels at the tape and at the glare
$W = \sum_j w_j$ over each group, with $w_j = \max(I_j - \bar{I} - 0.15,\ 0)$ the weight of one pixel; the labels $\ell, g$ are ours



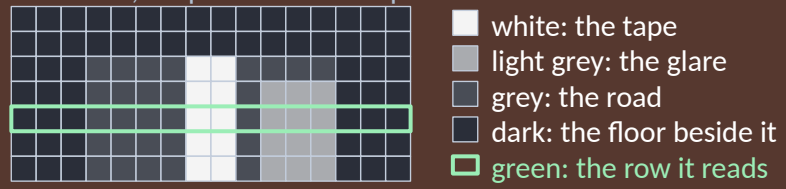

**TALK · 2 MIN** How would you explain, in your own words, why a rule that works on one floor can fail on another? Then, in the plot above, why does the estimate land between the line and the glare?

## THE DETECTOR ON 300 FRAMES

| the scene | mean error $\lvert\hat{x} - x\rvert$ |
|---|---|
| along the line | 0.007 |
| beside the line | 0.021 |
| with glare | 0.083 |
| the agreed error | 0.03 (3 pixels) |

$\hat{x}$ found, $x$ true column of the line, −1 to +1 across the frame

the frame, simplified to coarse pixels




*The detector reads the green row, keeps the pixels brighter than a threshold and returns the mean of their columns. On a clean floor only the tape passes; with glare the light grey pixels pass too and pull the mean toward them.*

# Convolutional Network

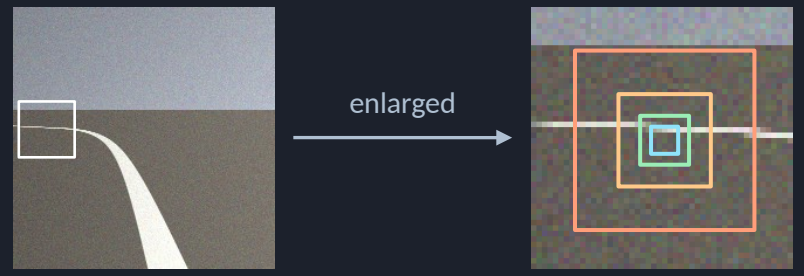
**Convolutional network** — a network whose layers slide small kernels, a few pixels wide, over the whole frame. Each kernel takes a weighted sum of the pixels under it, so it finds the same pattern wherever it lies. The weights are learned from labelled frames, not set by hand, and each layer combines the one before over a wider window.

## RULES SET BY HAND AGAINST LEARNED KERNELS

| | conventional vision | convolutional network |
|---|---|---|
| the feature | brighter than the floor | learned from labelled frames |
| the kernel | set by hand, e.g. Sobel | learned, 8 to 32 per layer |
| what it reads | one band of rows | the whole frame |
| it fails on | what was not foreseen | what was not shown |

In L5.1 the same idea sorted weld radiographs into clean, crack and pit: a kernel set by hand found the edge of the crack, and a small network learned its own kernels. Here the picture is the floor, and the answer is where the line is.

## WHAT EACH LAYER SEES




| layer | output | sees | weights |
|---|---|---|---|
| 1, 5 × 5 | 112 × 112 × 8 | 5 px | 608 |
| 2, 3 × 3 | 56 × 56 × 16 | 9 px | 1 168 |
| 3, 3 × 3 | 28 × 28 × 32 | 17 px | 4 640 |
| 4, 3 × 3 | 14 × 14 × 32 | 33 px | 9 248 |
| dense | 32 | all | 200 736 |
| output | 1 number | all | 33 |

*216 433 weights, 93 % of them in the dense layer.*

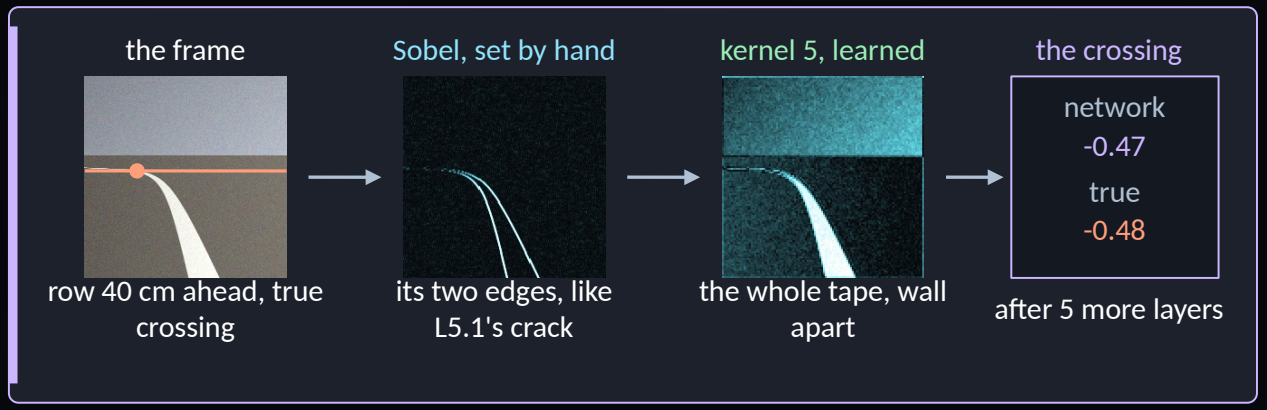
# Steering From a Frame

**Steering network** — a convolutional network that reads one camera frame and returns one number: where the taped line crosses the image row that sees the floor 40 cm ahead, from −1 at the left edge to +1 at the right. The car steers by that number times a gain. Its kernels are learned from frames whose true crossing is known.

## ONE CONVOLUTION, AS IN L5.1

$$y_{ij} = \sigma\Big(\sum_{m,n} w_{mn}\, x_{i+m,\, j+n} + b\Big)$$

$x$ the frame, $w$ the kernel, $b$ its bias, $\sigma$ the ReLU, $y$ the feature map




## DISTANCE FROM THE TRUE CROSSING

| 300 test frames | detector | network |
|---|---|---|
| on the line | 0.8 cm | 1.8 cm |
| beside the line | 4.5 cm | 3.6 cm |
| with glare | 14.1 cm | 5.3 cm |
| largest miss, glare | 79 cm | 32 cm |

*Root mean square, across the floor 40 cm ahead; lower is better. The truth is exact: the twin draws each frame from a known pose and computes where the tape crosses that row. On the line the hand-set detector is closer; beside it and with glare the network is. Its price: 80 s of training and twice the time per frame.*

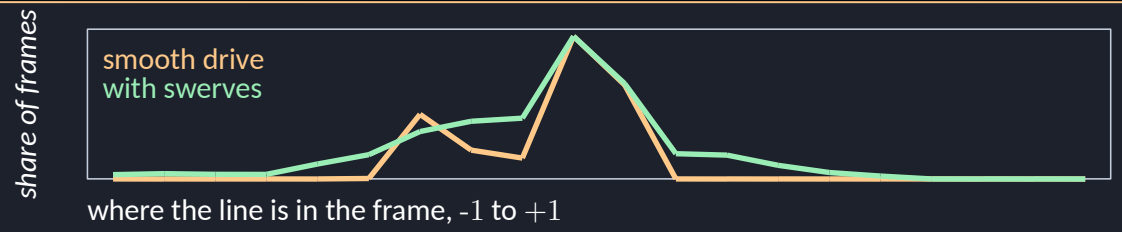
# Learning From a Human Driver

**Training drive** — a person drives the car round the course with a gamepad while the camera saves a frame ten times a second, each labelled with where the line truly is. Like the survey cars that drive every road once to build a map, the drive is done once, before training. The network learns only what this drive shows.

## THE LOSS IS AN AVERAGE OVER THE DRIVE'S FRAMES

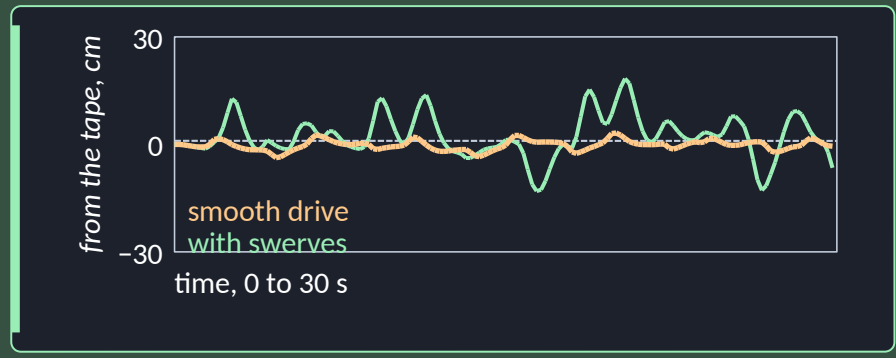
$$\mathcal{L} = \frac{1}{N}\sum_{k=1}^{N}(\hat{y}_k - y_k)^2$$

$N$ the frames of the drive, 1500; $\hat{y}_k$ the network's crossing in frame $k$, $y_k$ the true one




A driver who never leaves the line shows the network no way back to it.

## TWO TRAINING DRIVES




| mean error, test frames | smooth | swerves |
|---|---|---|
| car on the line | 0.010 | 0.021 |
| car beside the line | 0.195 | 0.036 |
| with glare | 0.194 | 0.050 |

*Trained on the smooth drive the network is 5 times worse beside the line, where a drifting car needs it. Same network, same 1500 frames: only the drive differs.*

# Why a GPU

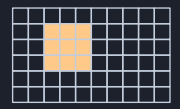
**GPU** — a processor with many small cores that run the same operation on different data at the same time. Every output of a convolution is its own sum of products over a small window, independent of the others, so a GPU computes them all at once.

## THE DISTANCE DRIVEN WITHOUT NEW INFORMATION

$$d = \frac{v}{f} = v\,(t_{\text{cap}} + t_{\text{pre}} + t_{\text{net}} + t_{\text{act}})$$

$d$ the blind distance, $v$ the speed, $f$ decisions per second
$t$ the times to capture, preprocess, run the network and act

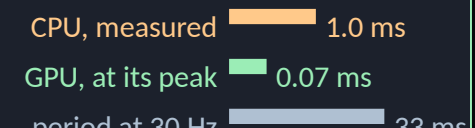
### THE SAME CONVOLUTION ON A CPU AND ON A GPU



CPU: one window after another



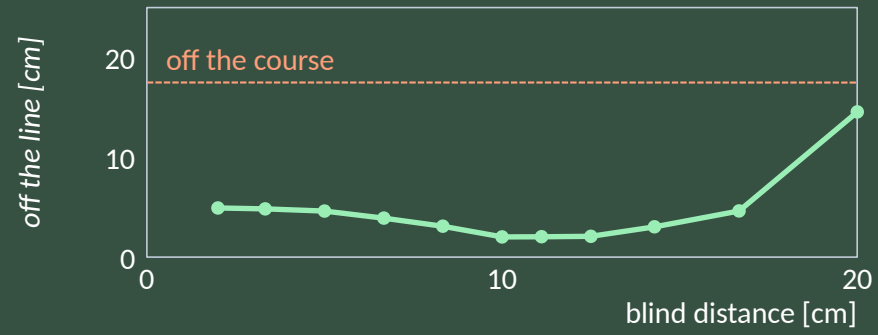
GPU: all windows at once




time for one frame, log scale

**TALK · 2 MIN** How would you explain, in your own words, why a slightly more accurate but slower network can be a worse driver? Then, in the plot on the right, where does the car start to swing wide?

## THE LAP, AND THE WORK PER FRAME




$$\text{MAC} = H_{\text{out}}\, W_{\text{out}}\, C_{\text{out}}\, C_{\text{in}}\, k^2$$

$H_{\text{out}}, W_{\text{out}}$ the output size, $C_{\text{in}}, C_{\text{out}}$ the channels, $k$ the kernel

| | |
|---|---|
| **one MAC** | a multiply and an add = 2 FLOP |
| **Ex_11's network** | 17 million MAC a frame |
| **laptop CPU, measured** | 0.9 ms a frame |
| **Nano GPU, at its peak** | 0.07 ms: 236 billion MAC/s |

# Advanced Topics

## RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

| topic | what it adds | read | where it makes a difference |
|---|---|---|---|
| semantic segmentation | which pixels are free to drive on | UDL ch. 10 | floors with no line |
| depth from one camera | distance from a single frame | UDL ch. 10 | no range sensor on board |
| vision transformers | attention over patches of the frame | UDL ch. 12 | large, varied data sets |
| imitation learning | a policy from human driving | UDL ch. 19 | road vehicles |
| reinforcement learning | a policy from reward, by trial | UDL ch. 19 | simulators, legged robots |

### WHY THEY ARE LEFT OUT

Ex_11 follows one taped line with one camera, scored on rendered frames where the true position is known, so none of them is needed to do it. They matter for the full-size machines of the first section.

# Exercise 11

## ONE NOTEBOOK · THIS LECTURE: SECTIONS 1 – 4 AND 7

- **1 – 3** — the virtual JetRacer, its frames, the detector
- **4** — a network that steers; TODO 1, the convolutions
- **7** — the loop rate and the GPU, this lecture's part
- **5 – 6, fewer bits** — left for L11.2: obstacles and strategies
- **8 – 11** — the race, the report and the mini projects



*the same task on the real JetRacer: a demonstration in the lab; the mini projects run on the Yahboom ROSMASTER R2*

### THE QUESTION EACH PART ANSWERS

| | |
|---|---|
| **1 – 3** | when is a fixed rule enough? |
| **4** | what must the training drive show? |
| **7** | how fast must the loop be? |
| **for L11.2** | sections 5 – 6, and the report |

### REPORT QUESTION, TO CONCLUDE

Which avoidance strategy reaches B fastest without a collision, and what limits its speed: the errors of the networks, the loop rate, or the distance needed to react?

SUMMARY

# Key Takeaways

## VISION POLICY

| | |
|---|---|
| **what it is** | a CNN: frame to where the line is |
| **function** | where the line is, a little ahead |
| **train, tune** | a human drive that swerves and back |
| **vs a fixed rule** | no hand tuning, holds under glare |
| **used for** | steering cars, robots, vehicles |

## THE LOOP ON A GPU

| | |
|---|---|
| **what it is** | many cores, one operation each |
| **function** | all outputs of a convolution at once |
| **train, tune** | the whole loop, measured on the car |
| **vs accuracy** | judged together, rate first |
| **used for** | a short blind distance $d = v/f$ |

*Next: what blocks the path - obstacles, and what to do about them.*

# Questions

## 1 – 2 · A NETWORK THAT STEERS

**1** **Why can a convolutional network steer from a camera frame where a line detector fails?**

The detector follows fixed rules, so anything bright pulls its aim. The network learns its own features from frames with the true position of the line. With glare the detector's error rose to 0.08, while the network's stayed at 0.05. *Go deeper: the worst frames of each (Ex_11 sections 3 – 4); UDL ch. 10.*

**2** **What must the training frames show for the network to bring a drifting car back?**

The floor, the light and the disturbances it will meet, and positions off the line, labelled with the way back. So the driver who records them swerves on purpose. Trained on a smooth drive, the network is about five times worse beside the line. *Go deeper: the smooth against the swerving drive (Ex_11 section 4); UDL ch. 8.*

## 3 – 4 · THE LOOP ON A GPU

**3** **Why is a steering network judged by the rate of the control loop as well as by its error?**

Between two decisions the car drives blind on its last command. The virtual car stays within 5 cm up to 17 cm blind and swings 15 cm at 20 cm, so a slightly more accurate network that halves the rate is usually a worse car. The whole loop is timed. *Go deeper: the loop rate (Ex_11 section 7); time each stage of the loop.*

**4** **Why does a convolutional network need a GPU to steer in time?**

Every output of a convolution is its own sum of products, independent of the others, so a GPU's many cores compute them at once. The exercise's network takes a millisecond on a laptop, inside the 33 ms frame; a ResNet-18 does a hundred times the work and fits only on a GPU. *Go deeper: count the multiply-adds (Ex_11 section 7); UDL ch. 10.*

NEXT

# L11.2 — Obstacle Avoidance

What blocks the path: a network that tells free from blocked, the strategies that steer around an obstacle, and the distance the car needs to react.

## BEFORE THE NEXT SESSION

| | |
|---|---|
| **read** | UDL ch. 10 |
| **run** | Ex_11's notebook, sections 1 to 4 |
| **bring** | your detector and network errors |
| **think** | what in your frames is an obstacle? |

*The real JetRacer drives the same task in a lab demonstration, with the throttle limited in software and one hand on the stop.*